

FlashAttention From Scratch

SC4064 GPU Programming
Team 12

The Attention Bottleneck

Transformer Attention

Attention mechanisms power modern LLMs and vision systems.

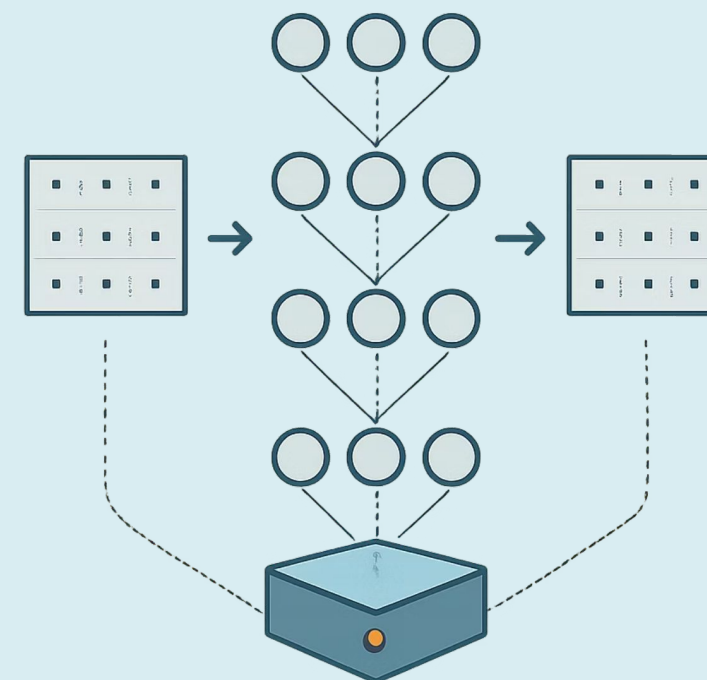
The scaled dot-product attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

Memory Challenge

Naive GPU implementation creates intermediate matrix, requiring quadratic memory growth with sequence length.

This leads to $\mathbf{O(N^2)}$ memory traffic, making the computation memory-bandwidth bound, preventing full utilization of Tensor Cores.



Related Work: Sparse Attention (BigBird)

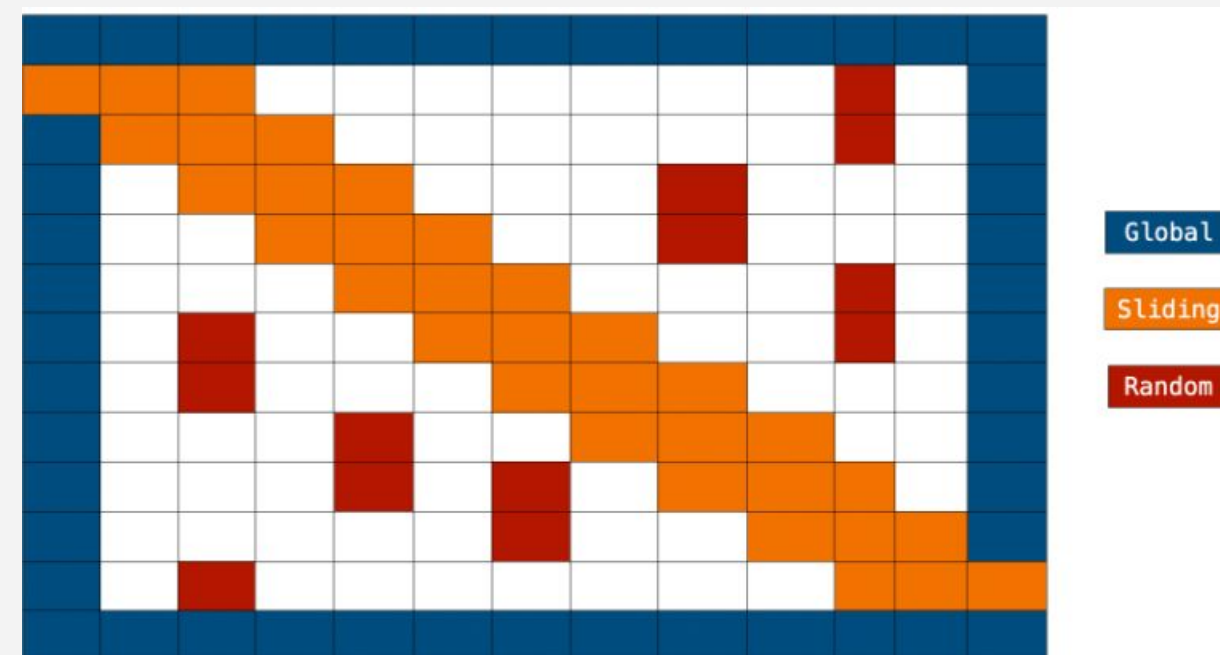
Core Idea:

Reduces cost by limiting token interactions.

Each token attends to subset instead of all tokens.

Uses three patterns:

- **Global tokens**
Attend to all tokens and receive attention from all
- **Sliding window**
Local context within a fixed window
- **Random connections**
Capture long-range dependencies



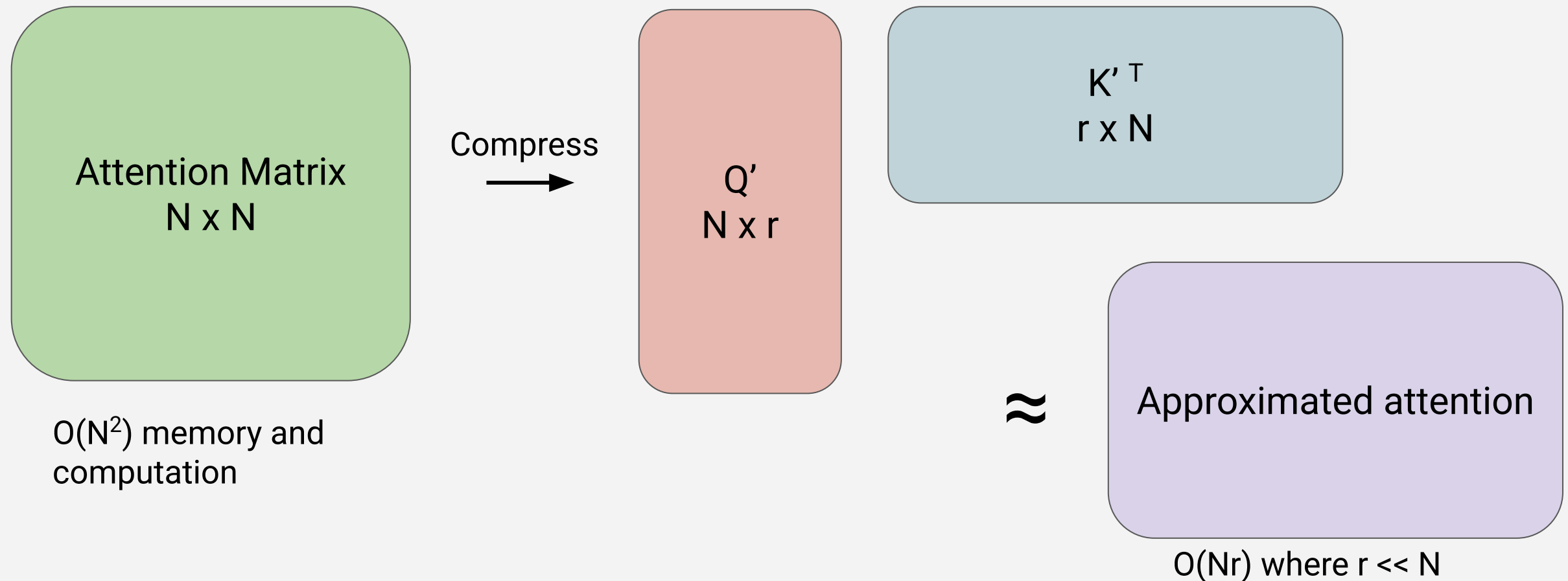
Benefits

- Reduces complexity from $O(N^2) \rightarrow O(Nk)$
- Significantly lower memory and compute
- Enables much longer sequences

Tradeoff

- Reduces cost by **removing interactions**

Related Work: Low-rank Attention



Core Idea:

Reduces cost by approximating the attention matrix

Benefits

- Reduces complexity from **$O(N^2)$** $\rightarrow$ **$O(Nr)$**
- Lower memory usage
- Less data movement

Tradeoff

- Reduces cost in exchange for **accuracy**

Flash Attention

Core Idea:

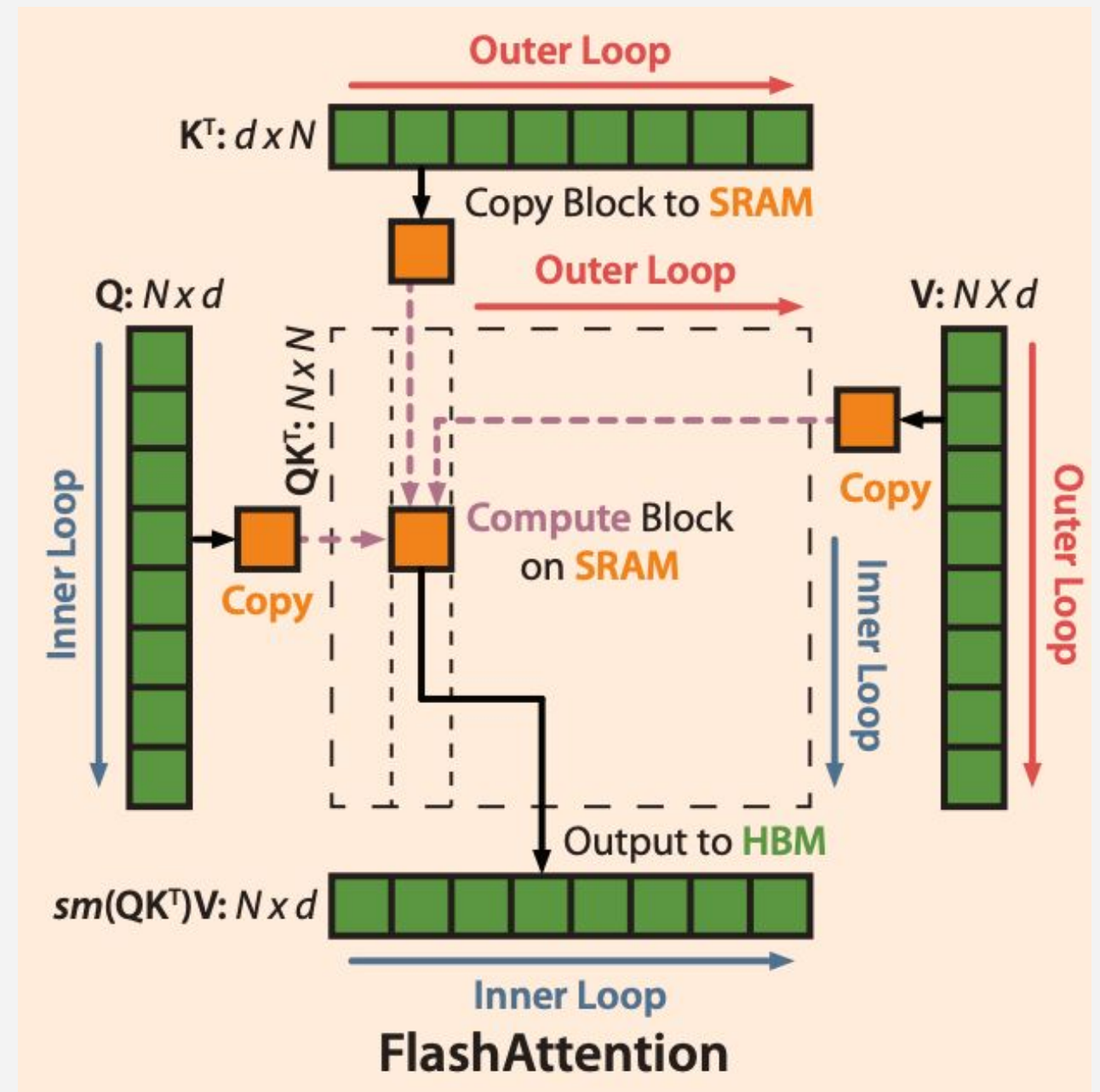
- Compute attention exactly, without approximations
- Reformulate computation to be IO-aware

Key Technique

- Avoid materializing $N \times N$ attention matrix
- Compute attention in small tiles
- Keep data in on-chip memory (shared memory / registers)

Advantages

- Preserves exact attention computation
- No information loss
- Achieves high performance through memory optimization



Our Solution: FlashAttention

We implemented FlashAttention through 11 progressive optimization stages, each building on the previous while maintaining exact attention computation.



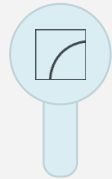
Naive Baseline

3-kernel implementation



Kernel Fusion and Tiling

Single kernel with online softmax and cooperative tiling eliminates intermediate HBM materialization.



Tensor Cores

WMMA API utilization with larger tiles uses hardware MMA units for a ~17x speedup.



Memory Hierarchy Optimization

Asynchronous double-buffering and shared memory padding resolve bank conflicts and hide global memory latency.



Advanced Layouts & Occupancy

CuTe swizzled layouts and tuned block dimensions ensure conflict-free access and optimal SM occupancy.



Custom PTX Implementation

Full kernel reimplementaion from scratch using raw CUDA/PTX

Optimization Techniques Applied

Kernel Fusion and Tiling

Collapsed 3 kernels into 1 with online softmax; added cooperative tiling



Tensor Cores via WMMA

Replaced scalar FP16 MADs with WMMA API on 16×16 fragments; doubled tile size



FA-2 Deferred Division

Moved softmax denominator division to epilogue step

Asynchronous Double-Buffering

Used cp.async for overlapping memory operations with computation



Shared Memory Padding

Eliminated bank conflicts by padding 64-element FP16 rows → 72 elements



CuTe Abstractions

Used Swizzle<3,3,3> → bank-conflict-free access; kept accumulator register-resident.

Q Hoisting & Swizzled Transposed V

Reduced redundant loads and eliminated column-major access conflicts



2-Block SM Occupancy with Idmatrix

Hid MMA latency; replaced scalar copies with warp-cooperative Idmatrix loads



Custom PTX Library

Reimplemented full kernel infrastructure in raw CUDA/PTX

Hardware & benchmark configuration

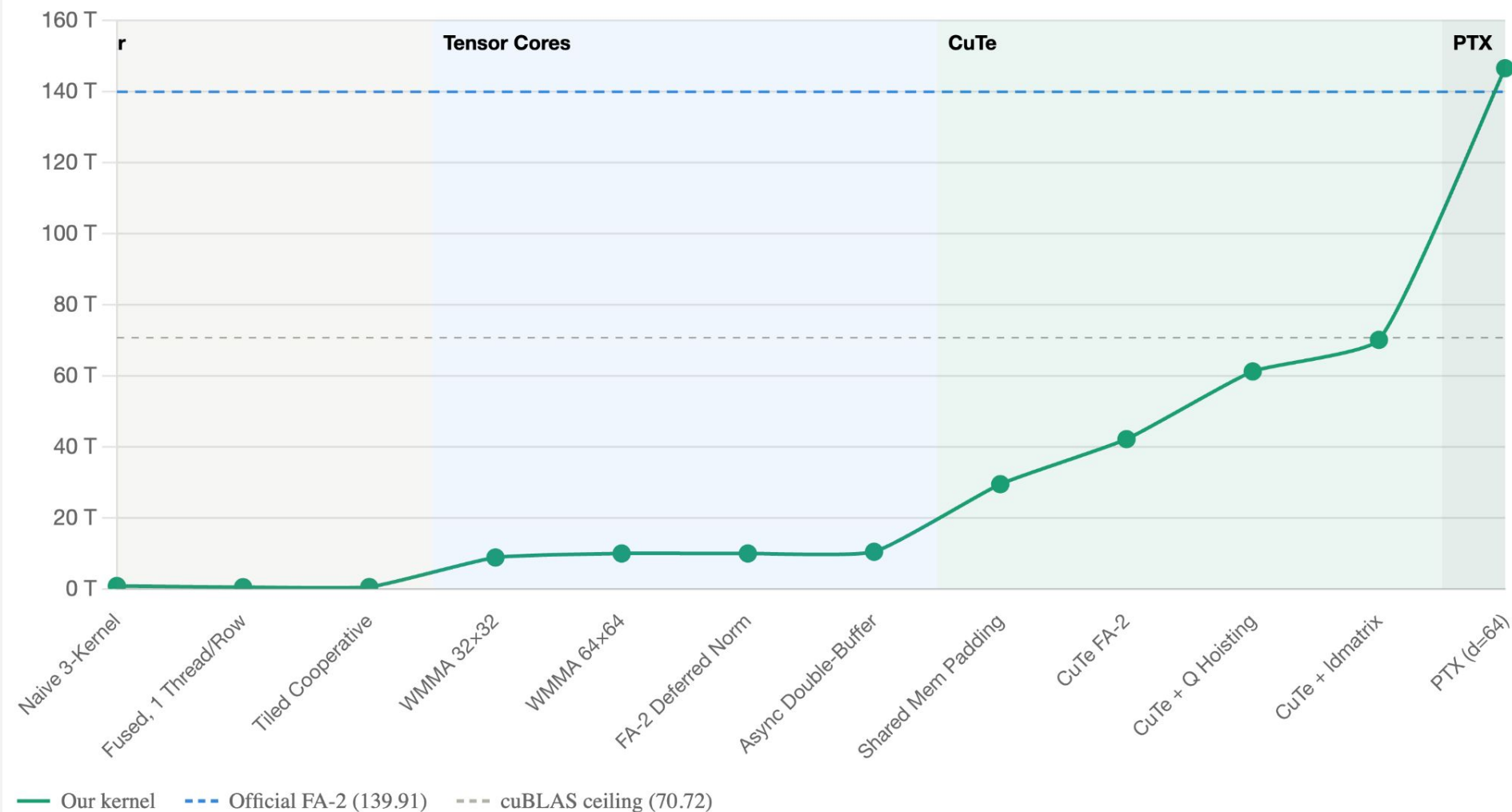
GPU	d=64 config	d=128 config
• Model: NVIDIA A100-SXM4-40GB	B=2, nh=16, B_nh=32	B=4, nh=16, B_nh=64
• Architecture: Ampere (sm_80)	BR=128, BC=64	BR=128, BC=64
• HBM2e bandwidth: ~2 TB/s	4 warps · 128 threads	4 warps · 128 threads
• SRAM per SM: 164 KB shared mem	2 blocks/SM · 32 KB smem	1 block/SM · 64 KB smem
• Tensor Core: mma.m16n8k16	load_0_0_0 fragments	load_2_2_2 fragments
• CUDA version: 12.2.2		

$$N \in \{1024, 2048, 4096\} \cdot 10 \text{ warmup iters} \cdot 100 \text{ timed iters} \cdot \text{TFLOPS} = 4 \times B_nh \times N^2 \times d / \text{time}$$

Stage-by-Stage Performance

All stages benchmarked at N=4096, d=64, on NVIDIA A100

FlashAttention TFLOPS Progression (N=4096, d=64)



- **+10.8× at S3 — Tensor Cores (wmma)**
- **+2.8× at S7 — bank conflict fix (CuTe Library)**
- **+2.1× at S11 — PTX pipelining**
- **179× total · S0 → S11**

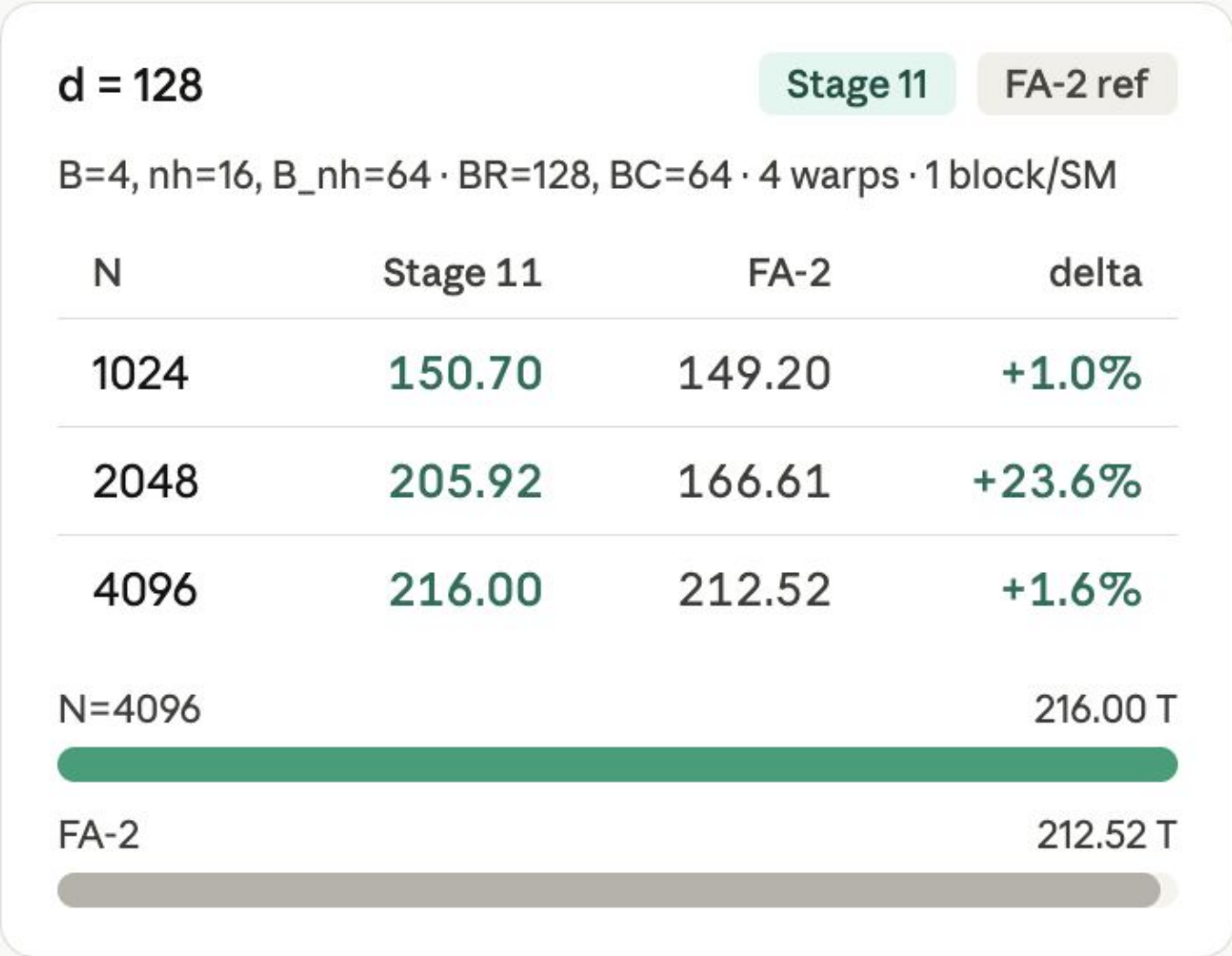
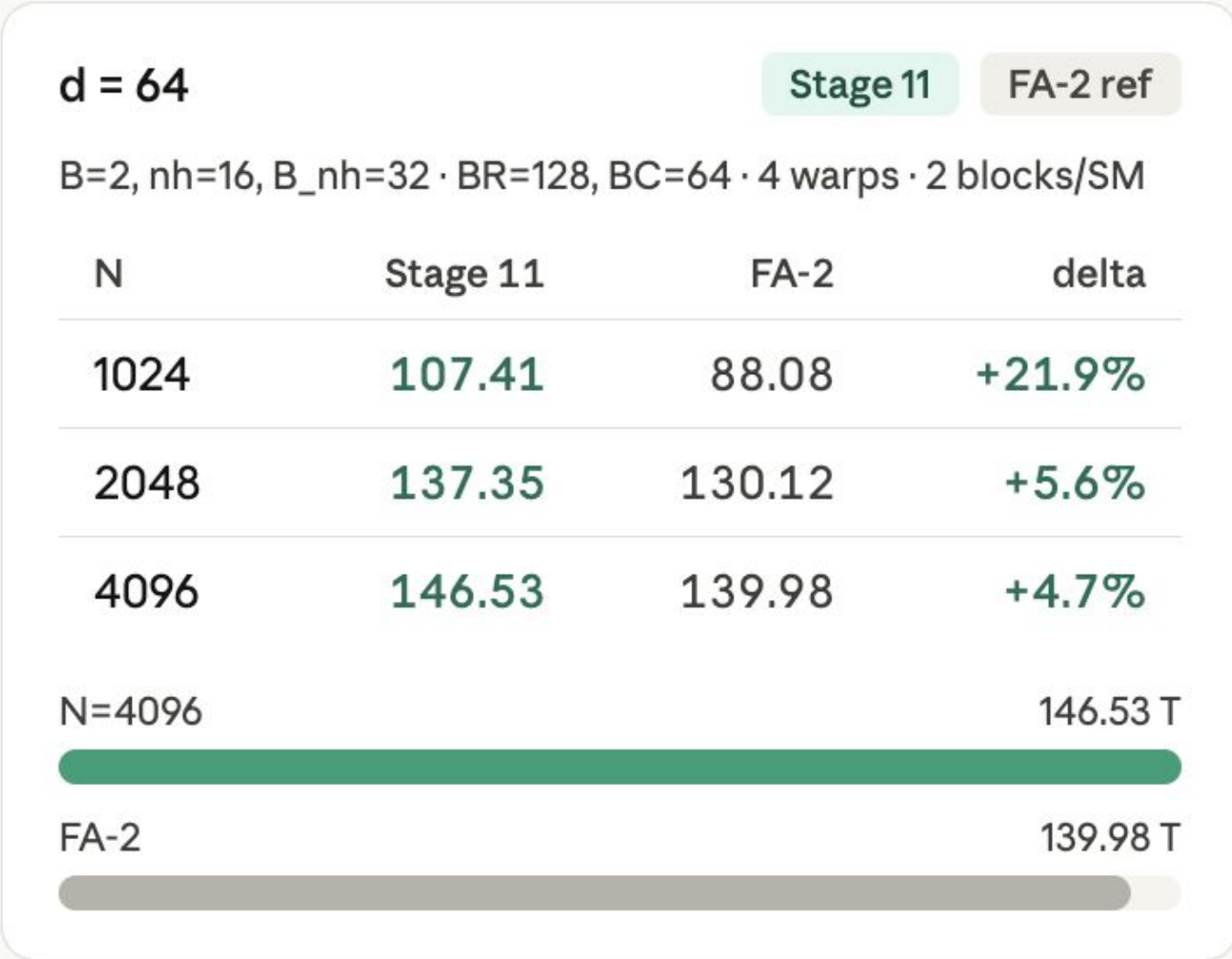
Stage-by-Stage Performance (Table)

Stage	Name	Time (ms)	TFLOPS
0	Naive 3-Kernel	168.059	0.82
1	Fused, 1 Thread/Row	272.778	0.50
2	Tiled Cooperative	253.844	0.54
3	WMMA Tensor Cores 32×32	15.552	8.84
4	WMMA 64×64, 8 Warps	13.779	9.97
5	FA-2 Deferred Normalisation	13.779	9.97
6	Async Double-Buffering	13.152	10.45
7	Shared Memory Padding	4.664	29.47
8	CuTe FA-2	3.262	42.15
9	CuTe + Q Hoisting + Swizzled V^T	2.246	61.29
10	CuTe + ldmatrix + 2 Blocks/SM	1.961	70.09
11	PTX from Scratch (d=64)	0.938	146.53
—	Official FlashAttention-2 (d=64)	0.982	139.91
—	cuBLAS ceiling	1.944	70.72

All stages benchmarked at
N=4096,
d=64,
NVIDIA A100 40GB

Stage 11 vs FA-2 Head-to-Head

All stages benchmarked at N=4096, d=64, on NVIDIA A100



Correctness Validation

- Verified against naive fp32 three-kernel reference at N=4096
- Small numerical differences expected from fp16 accumulation order
- Cosine similarity of 1.000000 across all stages confirms numerical correctness

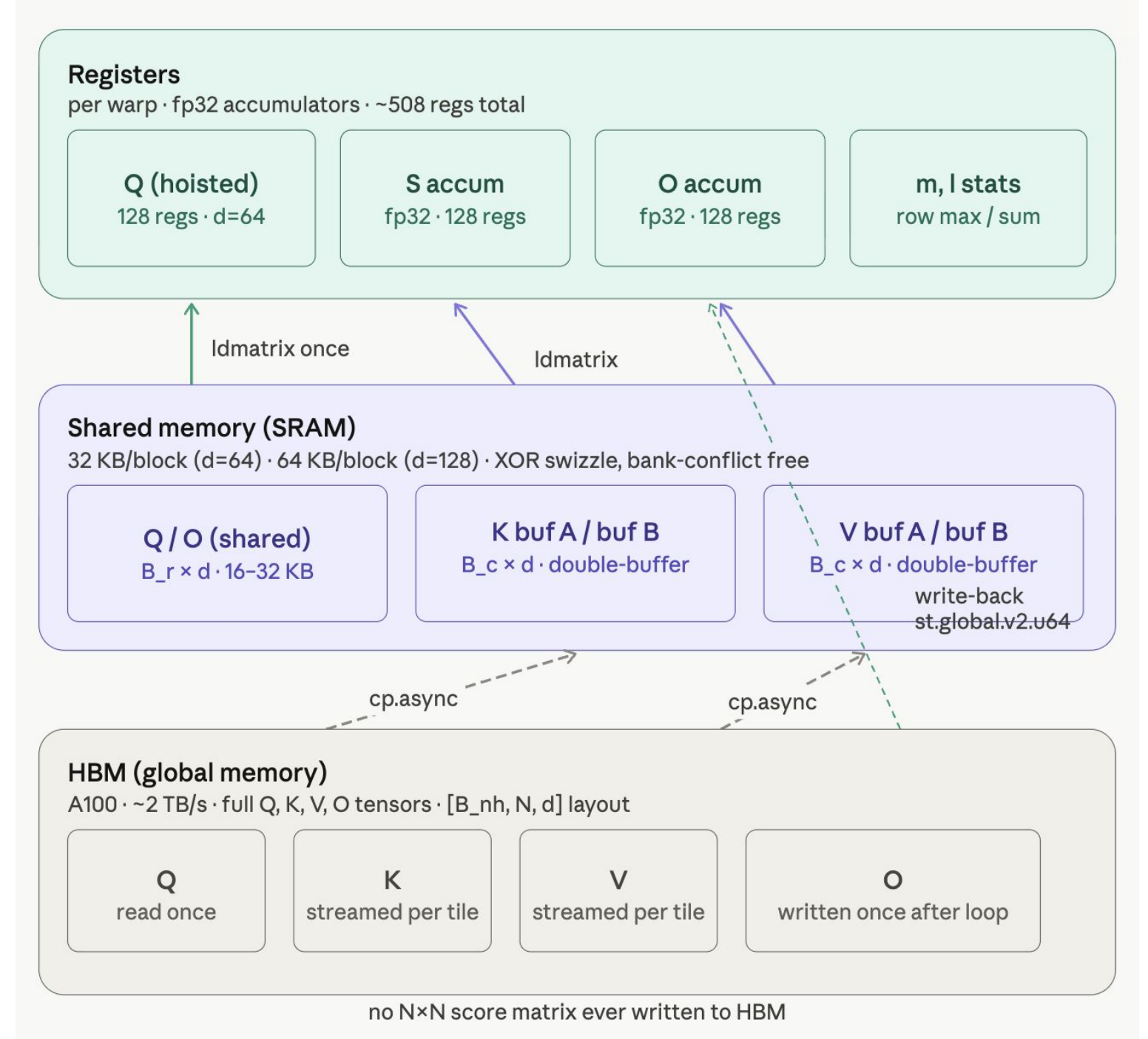
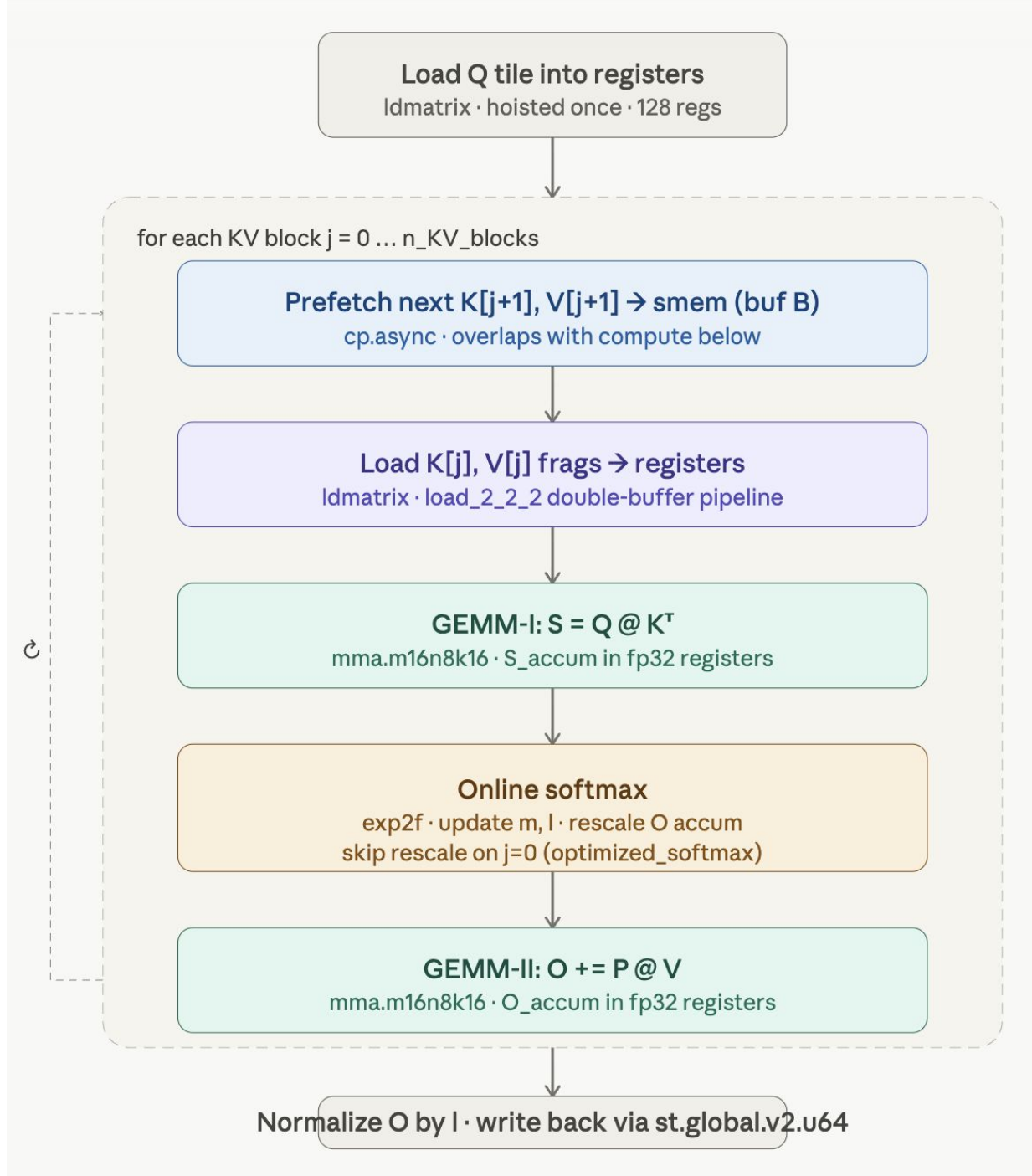
Stage	Max abs error	Mean rel error	Cosine similarity
S9 — CuTe + BR=128	3.05e-05	2.32e-03	1.000000
S10 — Idmatrix + 2 blk/SM	3.05e-05	2.52e-03	1.000000
S11 — PTX (d=64)	3.05e-05	2.31e-03	1.000000
S11 — PTX (d=128)	3.05e-05	2.40e-03	1.000000

All stages compared against naive fp32 three-kernel reference · N=4096 · A100

Conclusion

- Built FlashAttention from scratch across 12 stages, **achieving 146.53 TFLOPS (d=64) and 216.00 TFLOPS (d=128)** — exceeding official FA-2 at every sequence length
- **Attention is memory-bandwidth bound by default** — tiling eliminates the $N \times N$ bottleneck and is the single most impactful algorithmic change
- **Tensor Cores** give an immediate 10.8× jump, but a single **shared memory padding** fix (Stage 7) more than doubled performance again — memory layout matters as much as the algorithm
- **Q hoisting, cp.async double-buffering, and swizzled smem** layouts are essential to keeping the MMA pipeline fed — latency hiding is as important as peak compute
- High-level abstractions like **CuTe get you most of the way (near Cublas ceiling)**, but the final 2.1× required dropping to raw PTX where register pipelining is fully in your hands

PTX (Stage 11) Implementation



References

<https://huggingface.co/blog/big-bird> (sparse attention bigbird)

<https://pub.towardsai.net/flash-attention-underlying-principles-explained-44804a9e25ce> (Flash Attention)